



同濟大學
TONGJI UNIVERSITY

计算机系统实验申优答辩 ——操作系统移植和应用程序开发



2351520

毛星博

计算机科学与技术



增加总线控制器

- 为 **OpenMIPS** 增加 **Wishbone** 总线接口
- 增加 **Wishbone** 总线
- 增加 **GPIO**
- 增加 **UART** 控制器
- 增加 **Flash** 控制器
- 增加 **SDRAM** 控制器
- 实现完整的 **SOPC**

操作系统移植

- 在 **Ubuntu** 上建立交叉编译环境
- 对 **μC/OS-II** 系统进行改写与编译

应用程序开发

- 编写应用程序
- 使用串口进行交互



同濟大學
TONGJI UNIVERSITY

PART 1

实验二——操作系统移植

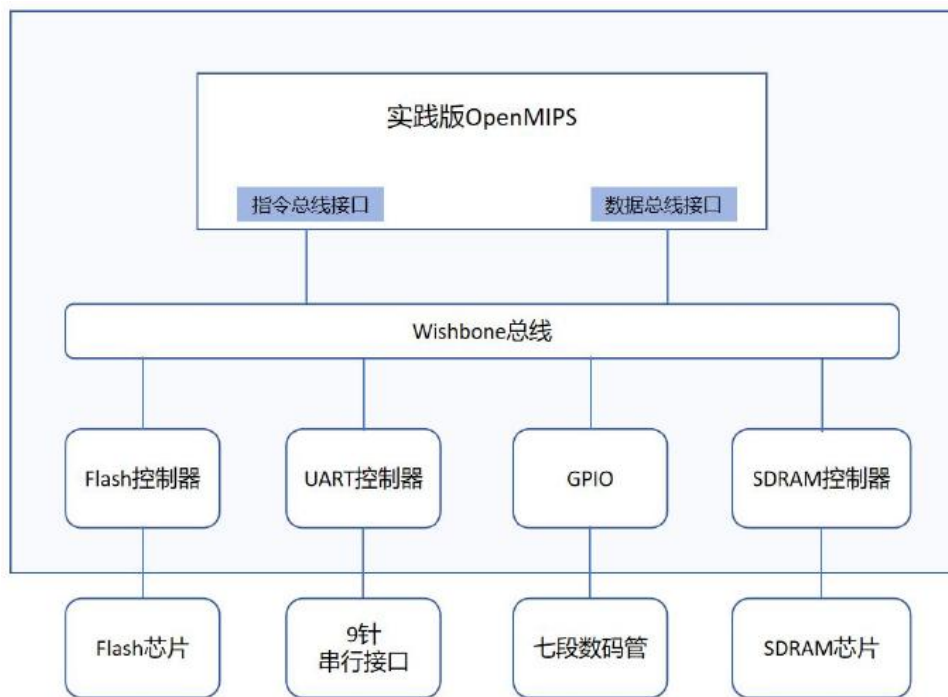
Experiment 2 - Operating System Porting

μC/OS-II 移植与下板验证

OpenMIPS / Wishbone / BootLoader / DDR2



从 CPU 到 SOPC



总线

系统扩展目标

以实验一 89 条指令 OpenMIPS 为核心，通过 Wishbone 总线统一连接外设与存储模块。

扩展模块：Flash、UART、GPIO、DDR2
SDRAM

运行目标：BootLoader + μ C/OS-II 能够在板上启动。

CPU 访问外设的统一入口



接口职责

- 将 CPU 侧访存请求转换为 Wishbone 事务
- 用 `cyc/stb` 发起访问，用 `ack` 判断结束
- 等待期间向流水线发出 `stallreq`，避免请求被覆盖

```

cpu_addr_i / cpu_data_i / cpu_we_i
    ↓
wishbone_addr_o / data_o / we_o
wishbone_stb_o + wishbone_cyc_o
    ↓
wishbone_ack_i -> cpu_data_o
    
```

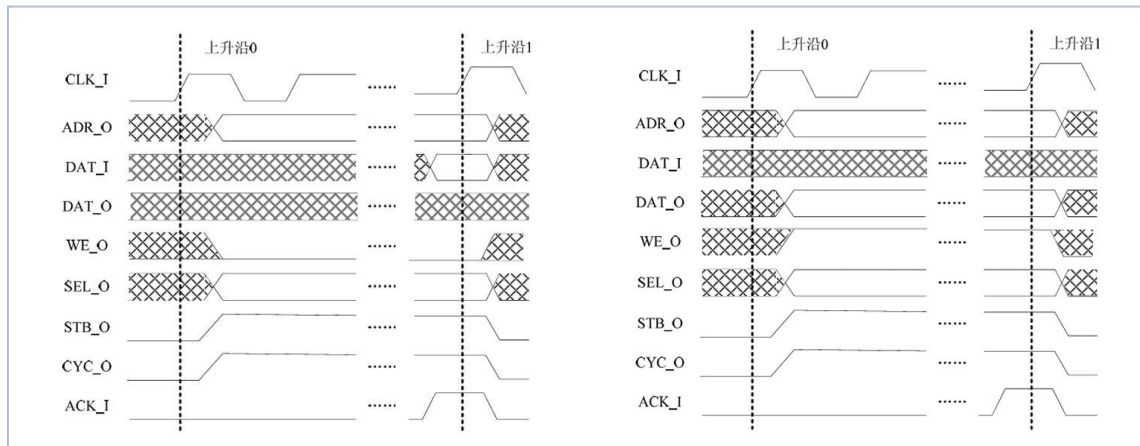
一次操作由请求和应答闭合

单次读

主设备给出地址 ADR_O
 $WE_O = 0$ 表示读操作
 CYC_O / STB_O 拉高
从设备返回 DAT_I 与 ACK_I

单次写

主设备给出地址和 DAT_O
 $WE_O = 1$ 表示写操作
 SEL_O 指明有效字节
从设备锁存数据并返回 ACK_I



意义

- OpenMIPS 不直接面对每个外设的复杂控制信号
- 指令通道和数据通道各实例化一个 wishbone_bus_if
- 统一握手让 Flash、UART、GPIO、SDRAM 都能接入系统

四类从设备接入 Wishbone

GPIO

读取开关 / 驱动七
段数码管

$\text{addr}[31:28] = 0x2$

UART16550

串口输出系统运行
信息

$\text{addr}[31:28] = 0x1$

SPI Flash

存放 BootLoader
和 OS.bin

$\text{addr}[31:28] = 0x3$

DDR2 SDRAM

系统运行时程序与
数据空间

$\text{addr}[31:28] = 0x0$

地址译码

- WB_CONMAX 作为共享总线互连矩阵
- Master0 连接数据通道, Master1 连接指令通道
- 不同高位地址选择不同外设或存储器



BootLoader 交接控制权



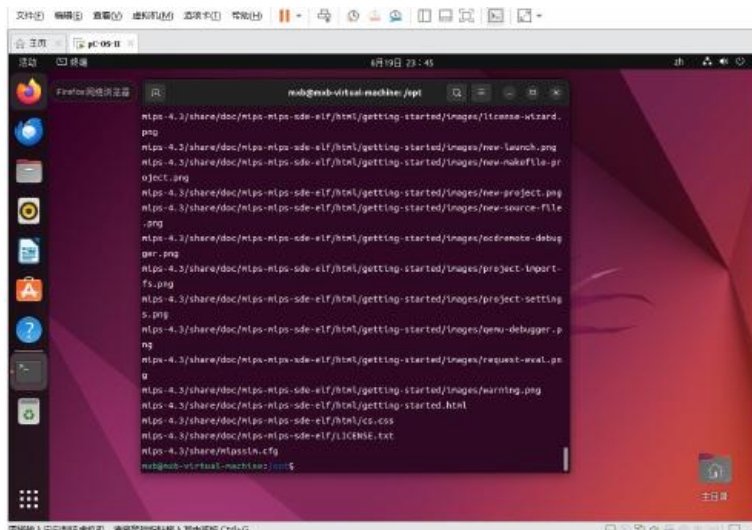
启动链路

上电后先运行 BootLoader。由于 Flash 速度较慢且只读，BootLoader 将 $\mu\text{C}/\text{OS-II}$ 镜像搬运到 DDR2 SDRAM，再跳转到 SDRAM 执行，完成控制权交接。

Ubuntu 中生成可下板镜像

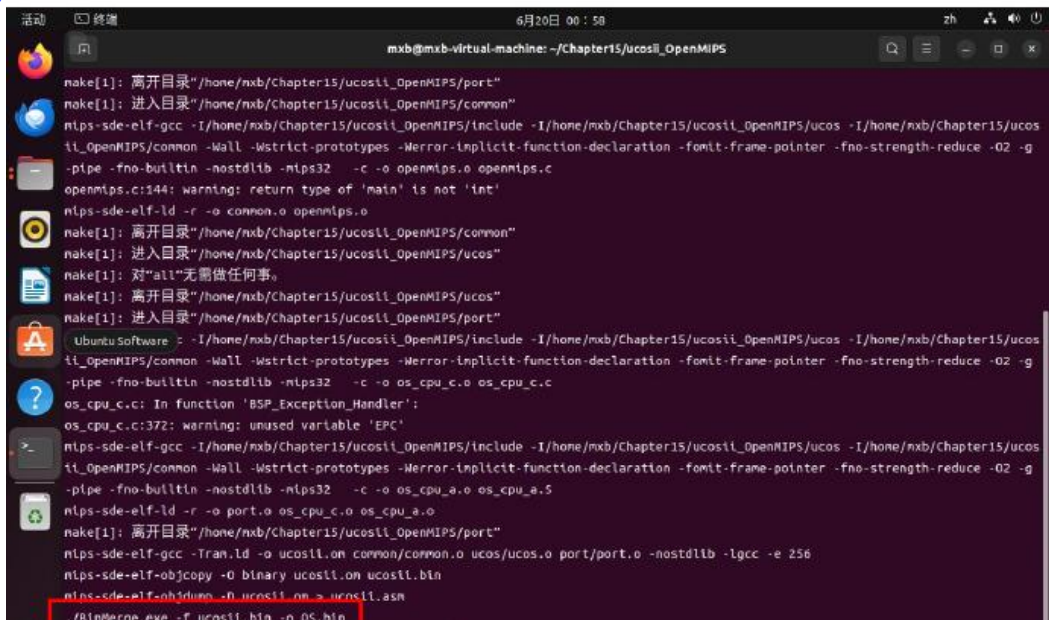
make

编译如下:



用户主目录 Home 文件夹，在窗口菜单栏中选择 View->Show Hidden Files，这样可以找到一个隐藏文件.bashrc，在此文件的最后加入

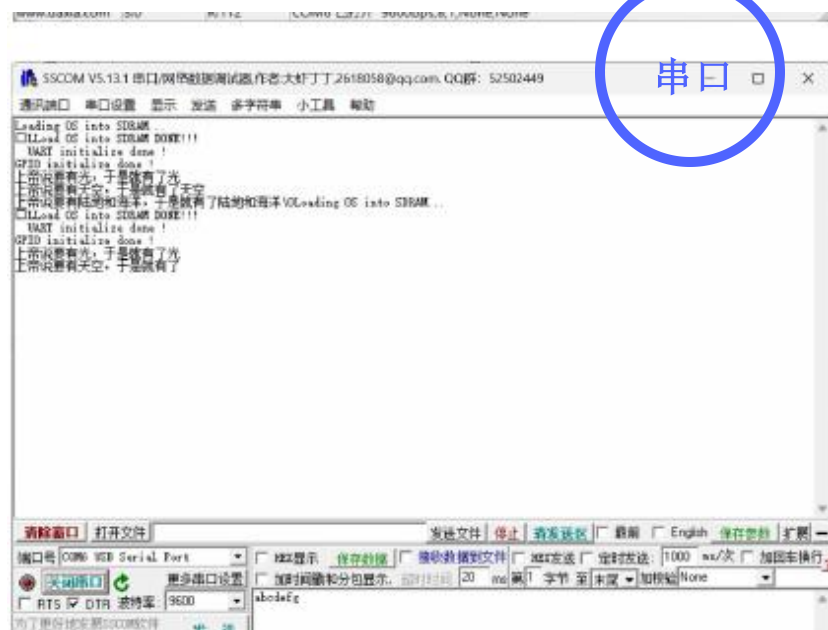
MIPS GNU 工具链安装完成



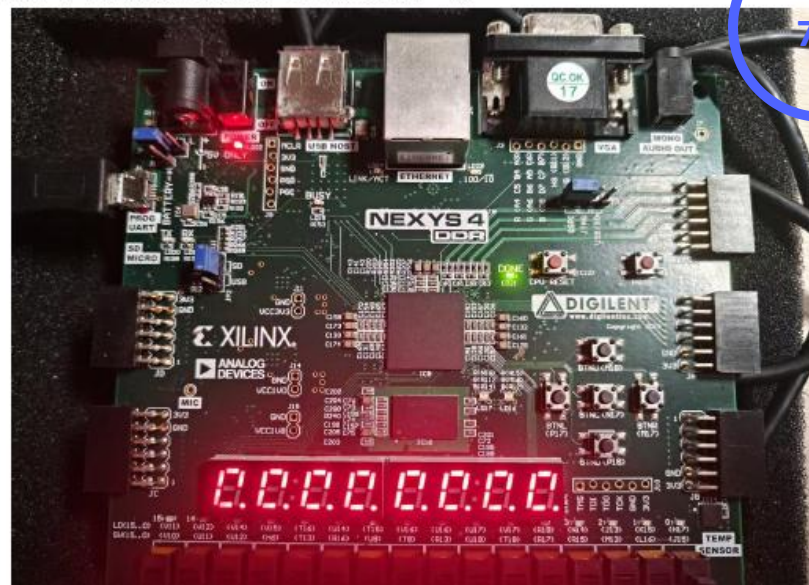
编译 BootLoader 与 μ C/OS-II，最终生成 OS.bin



串口输出与数码管验证



结果：7 段数码管的显示 100ms 变化一次



验证现象

- SSCOM 9600 baud 显示 BootLoader 加载与系统输出
- 七段数码管约每 100ms 变化一次
- 说明 $\mu\text{C}/\text{OS-II}$ 已在 OpenMIPS 平台上运行



同濟大學
TONGJI UNIVERSITY

PART2

实验三——应用程序开发

Experiment 3 - Application Development

可选难度走迷宫小游戏

OpenMIPS / μ C/OS-II / PuTTY 终端交互



游戏整体逻辑

```
// 1代表墙壁(#), 0代表通路(.)
const int maze_easy[10 * 10];
const int maze_medium[15 * 15];
const int maze_hard[20 * 20];
const int maze_hell[30 * 30];
```

```
const int *active_map;
int board_size;
int exit_row;
int exit_col;
int player_row;
int player_col;
```

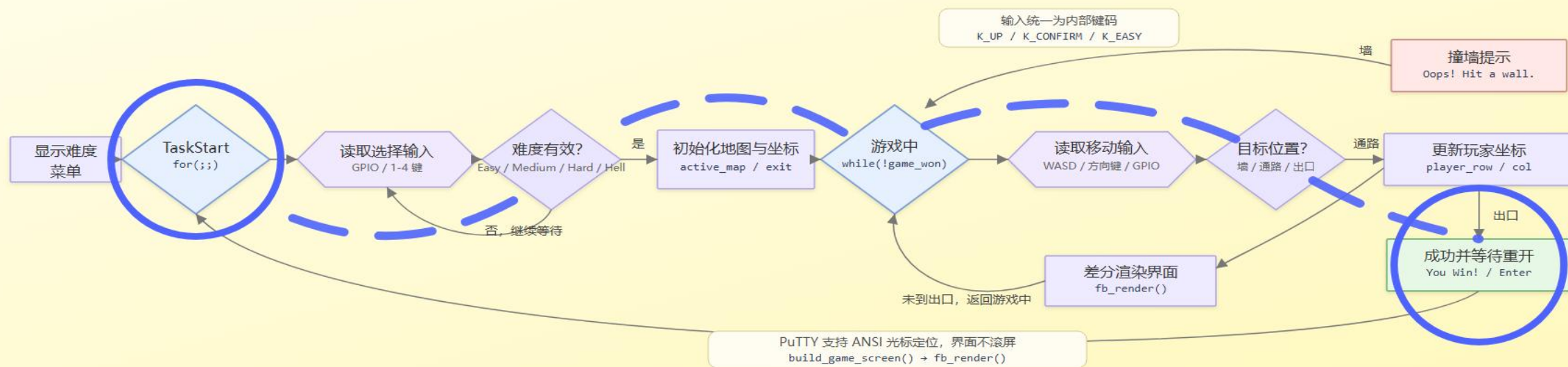
Easy 10x10 Medium 15x15

Hard 20x20 Hell 30x30

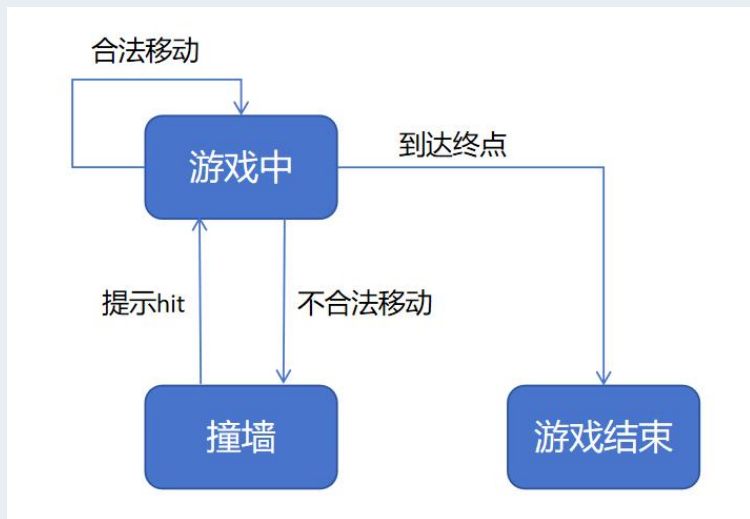
player '@' exit '\$' wall '#' road '.'

地图、玩家坐标、出口坐标和当前难度共同描述一局游戏。
不同难度只是地图规模不同，移动、撞墙、通关逻辑完全复用。

走迷宫小游戏主循环状态图 (TaskStart 任务)



按照游戏实际运行逻辑，游戏状态包括：游戏中、撞墙提示和成功。玩家处于游戏中时，如果移动到通路，就仍然保持游戏中；如果目标位置是墙壁，则进入“撞墙提示”状态，显示 Oops! 后立即重新渲染并返回游戏中；如果玩家到达出口，则进入成功状态，显示 You Win 并等待重开。



```
#####
# . . . # . . @ . #
# # . . # . # . #
# . . . . . # . . #
# . # # # # . . # #
# . # . . . # . . #
# . # . # . # # . #
# . . . # $ . # . #
# # # # # . # . #
#####

WASD/Arrows:move | Enter/Space:confirm | N17+switches:GPIO
```

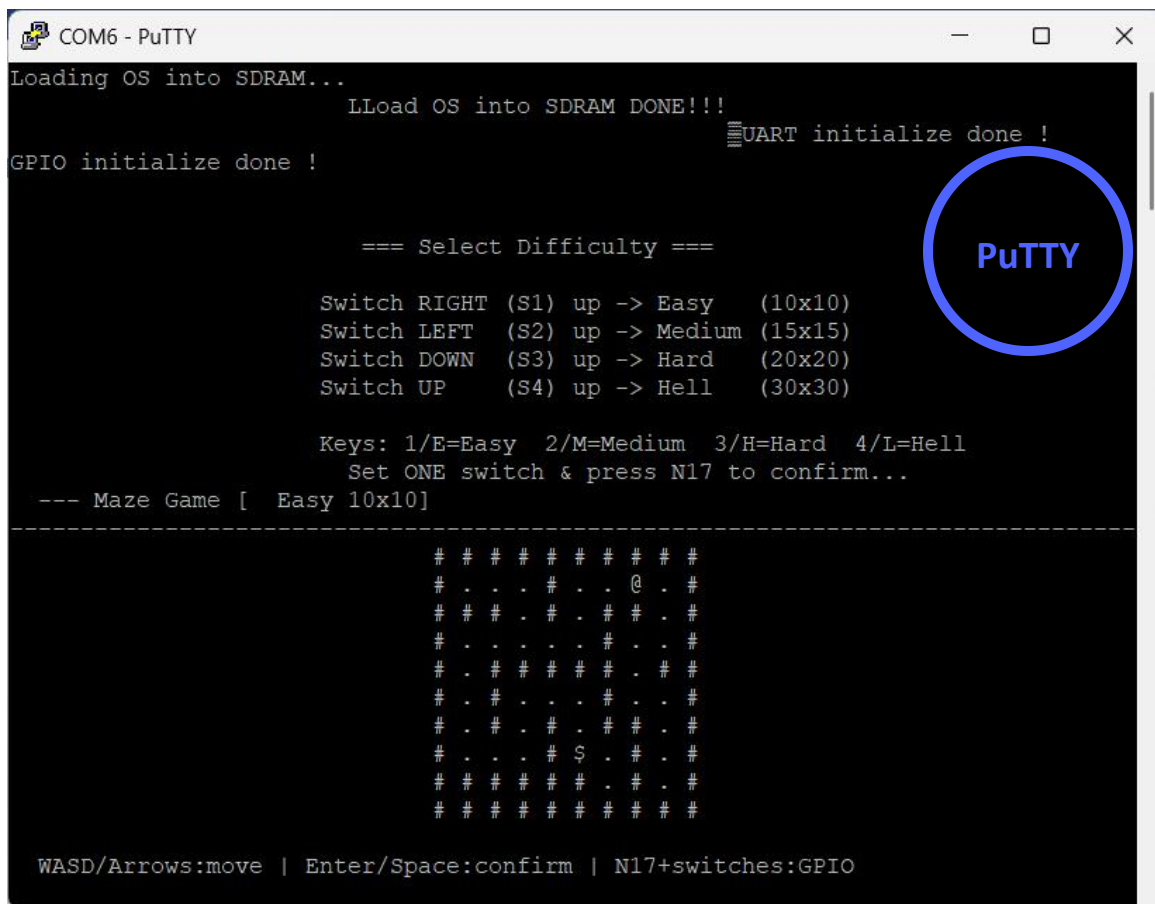
```
#####
# . . . # . . . . #
# # . . # . # # . #
# . . . . . # . . #
# . # # # # . . # #
# . # . . . # . . #
# @ # . # . # # . #
# $ . . # . . # . #
# # # # # . # . #
#####

Oops! Hit a wall.
WASD/Arrows:move | Enter/Space:confirm | N17+switches:GPIO
```

```
#####
# . . . # . . . . #
# # . . # . # # . #
# . . . . . # . . #
# . # # # # . . # #
# . # . . . # . . #
# @ . . # . . # . #
# # # # # . # . #
#####

You Win! Press N17 to restart...
```

PuTTY 原地刷新



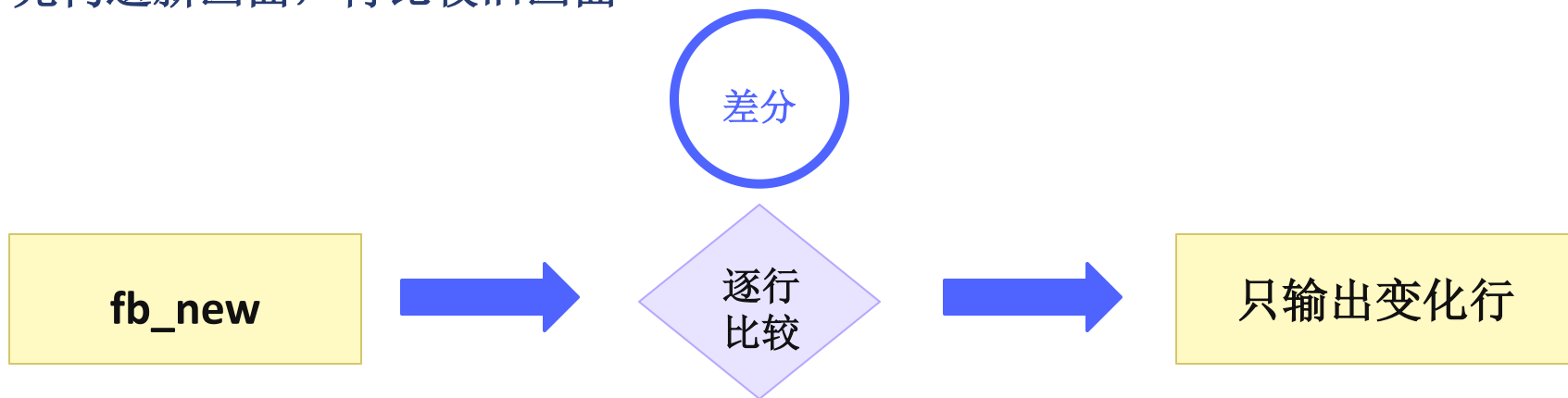
```
static void uart_ansi_pos(int row, int col) {  
    uart_putc_raw(27); // ESC  
    uart_putc_raw(['H');  
    ...  
    uart_putc_raw('H'); // ESC[row;colH  
}  
  
static void uart_send_str(const char *s) {  
    while (*s) uart_putc_raw(*s++);  
}
```

为什么不再使用 SSCOM

- SSCOM 只追加日志，原地刷新效果差
- PuTTY 支持 ANSI 清屏与光标定位
- 移动后只重画变化区域

帧缓冲与差分渲染

先构造新画面，再比较旧画面



差分效果

- 终端不会持续滚屏
- 角色移动只影响相关行
- PuTTY 界面接近字符游戏

```
if (my_memcmp(fb_new[r], fb_old[r], FB_COLS) != 0) {  
    uart_ansi_pos(r + 1, 1);  
    for (c = 0; c < FB_COLS; c++) {  
        uart_putc_raw(fb_new[r][c]);  
    }  
}  
my_memcpy(fb_old[r], fb_new[r], FB_COLS);
```

GPIO 与键盘共存

 输入

输入映射关系

GPIO 模式
方向开关 + N17 确认

键盘模式
WASD / 方向键移动
1 2 3 4 选择难度
Enter / Space 重开

uart_get_key()

非阻塞读取 UART 字符，并把不同按键统一映射为内部键码。方向键不是单字符，而是 ESC [A/B/C/D 序列，因此代码用 `esc_state` 分步解析。

```
if (c == 'w' || c == 'W') return K_UP;  
if (c == 'a' || c == 'A') return K_LEFT;  
if (c == '1' || c == 'E') return K_EASY;  
if (c == '\r' || c == ' ') return K_CONFIRM;  
  
if (esc_state == 2 && c == 'A') return K_UP;
```

地图生成函数采用深度优先搜索 DFS，递归回溯生成迷宫的非递归版本，因为 OpenMIPS / 嵌入式环境里不太适合递归和动态内存，所以用固定数组当栈来模拟 DFS。

先把整张地图全部填成墙，也就是 1。然后从 (1,1) 开始，把它设为通路 0，再用 DFS 一步步“挖墙”：

每次取栈顶格子 (r, c)，然后把上下左右四个方向随机打乱：

如果目标格还没被访问过，也就是仍然是墙：

那就把中间墙和目标格都挖开：

这样就从当前位置连通到了目标格。

如果当前格子还能往某个方向挖，就把新格子压入栈：

如果四个方向都不能继续挖，说明当前路走到头了，就退回上一个格子：

```
static int generate_maze_map(int *maze, int size, INT32U seed)
{
    int r, c, i, j;
    int top;
    int dirs[4];
    int carved;
    int d, nr, nc, mr, mc;
    const int dr[4] = {-1, 1, 0, 0};
    const int dc[4] = {0, 0, -1, 1};

    if (maze == (int *)0 || size < 5 || size > MAX_MAZE_SIZE)
    {
        return 0;
    }
    if (seed == 0)
    {
        seed = 1;
    }
    for (r = 0; r < size; r++)
    {
        for (c = 0; c < size; c++)
        {
            maze[r * size + c] = 1;
        }
    }
    top = 0;
    maze[1 * size + 1] = 0;
    maze_stack_r[top] = 1;
    maze_stack_c[top] = 1;
    top++;
    while (top > 0)
    {
        r = maze_stack_r[top - 1];
        c = maze_stack_c[top - 1];
        for (i = 0; i < 4; i++)
        {
            dirs[i] = i;
        }
        for (i = 3; i > 0; i--)
        {
            j = maze_rand_next(&seed) % (i + 1);
            d = dirs[i];
            dirs[i] = dirs[j];

            dirs[j] = d;
        }
        carved = 0;
        for (i = 0; i < 4; i++)
        {
            d = dirs[i];
            nr = r + dr[d] * 2;
            nc = c + dc[d] * 2;
            if (nr > 0 && nr < size - 1 && nc > 0 && nc < size - 1 &&
                maze[nr * size + nc] == 1)
            {
                mr = r + dr[d];
                mc = c + dc[d];
                maze[mr * size + mc] = 0;
                maze[nr * size + nc] = 0;
                maze_stack_r[top] = nr;
                maze_stack_c[top] = nc;
                top++;
                carved = 1;
                break;
            }
        }
        if (!carved)
        {
            top--;
        }
    }
    /* Make sure the default exit cell is reachable even when size is even. */
    maze[(size - 2) * size + (size - 2)] = 0;
    maze[(size - 2) * size + (size - 3)] = 0;
    maze[(size - 3) * size + (size - 2)] = 0;
    maze[(size - 3) * size + (size - 3)] = 0;
    return 1;
}
```




同濟大學
TONGJI UNIVERSITY

PART3

下板演示——走迷宫小游戏

下板验证应用程序

OpenMIPS / Wishbone / BootLoader / DDR2



同濟大學
TONGJI UNIVERSITY

致谢:

陪伴两年的NEXYS4 DDR Atrix-7开发板

郭玉臣老师、张东东老师、陆有军老师、蒋海鹰老师

亲爱的室友刘神和徐神

冯羽芯学姐、曾崇然学长



2351520

毛星博

计算机科学与技术



同濟大學
TONGJI UNIVERSITY

计算机系统实验申优答辩
——操作系统移植和应用程序开发

感谢!

致谢:

陪伴两年的NEXYS4 DDR Atrix-7开发板

郭玉臣老师、张东东老师、陆有军老师、蒋海鹰老师

亲爱的室友刘神和徐神

冯羽芯学姐、曾崇然学长

2351520

毛星博

计算机科学与技术

